



Render Scratch, Blockly, MakeCode and Open Roberta blocks for educational documents.

blockst renders programming blocks directly in Typst documents — Scratch, Blockly with a jwinf profile, MakeCode for the micro:bit and the Calliope mini, and Open Roberta NEPO — for worksheets, tutorials and teaching material. Block code is written as plain text and rendered by a bundled WASM plugin, Scratch in 26 languages including right-to-left scripts, with a turtle-graphics execution engine and helpers for importing real `.sb3` project files.

Inhalt

1	Introduction	3
1.1	About blockst	3
1.2	Quick start	3
1.3	Which function for which editor	4
1.4	Package information	5
2	The notation	6
3	Options for every editor	7
3.1	set-blockst() — Global Defaults	7
3.2	blockst() — Group Override Container	7
3.3	Themes	7
3.4	Colours	8
3.5	Fonts	9
3.6	Scale and inset scale	9
3.7	Line numbers	9
4	Scratch	10
4.1	scratch() — Render Scratch Blocks	10
4.2	raw-scratch — Code fences	10
4.3	Languages	11
4.3.1	Right-to-left languages	12
4.4	@category — Quick Color Defaults	13
4.5	scratch-parse() — Parse to AST	13
5	Blockly and jwinf	14
5.1	blockly() — Render Blockly Blocks	14
5.2	Profiles	14
5.3	Writing jwinf tasks	15
5.4	Languages	15
5.5	Code fences and parsing	15
6	MakeCode	16
6.1	makecode() — Render MakeCode Blocks	16
6.2	Profiles	17
6.3	Writing MakeCode programs	17
6.4	Languages	17
6.5	Code fences and parsing	17
7	NEPO (Open Roberta)	18
8	Labels and line numbers	19
8.1	blockst-labels() — Query Labels	19
8.2	scratch-labels() — Extract Labels	19
8.3	blockst-register-labels() — Pre-register Labels	19
9	Running Scratch programs	21
9.1	scratch-run Module	21
9.1.1	scratch-run.stage() — Stage Canvas	21
9.1.2	scratch-run.grid() — Coordinate Grid	21
9.1.3	set-scratch-run() — Run Global Defaults	21

9.1.4	Supported Execution Commands	22
9.1.5	Complete Example	23
10	Importing Scratch projects (SB3)	26
10.1	Basic Workflow	26
10.2	render-sb3-scripts()	26
10.3	render-sb3-variables()	27
10.4	render-sb3-lists()	27
10.5	Standalone Monitors	28
10.5.1	list-monitor()	28
10.5.2	variable-monitor()	28
10.6	screen-preview()	28
10.7	Image Helpers	28
10.8	Catalog Helpers	29
11	Scratch block catalog	30
11.1	Motion	30
11.2	Looks	31
11.3	Sound	33
11.4	Pen	34
11.5	Variables	35
11.6	Lists	35
11.7	Events	36
11.8	Control	37
11.9	Sensing	39
11.10	Operators	40
12	Contributing	42
13	API Reference	43
13.1	blockly	43
13.2	blockly-parse	44
13.3	blockst	44
13.4	blockst-labels	45
13.5	blockst-register-labels	45
13.6	makecode	45
13.7	makecode-parse	46
13.8	raw-blockly	46
13.9	raw-makecode	46
13.10	raw-scratch	46
13.11	scratch	47
13.12	scratch-execute	47
13.13	scratch-labels	47
13.14	scratch-parse	48
13.15	set-blockst	48

1 Introduction

1.1 About blockst

blockst renders block-based programs in Typst documents, each drawn the way its editor draws it:

- **Scratch 3** — `scratch()`, the Scratch look in 26 languages, including right-to-left scripts, with an execution engine for turtle graphics and helpers for importing `.sb3` project files.
- **Blockly** — `blockly()`, today's flat Blockly and the pre-2019 look, with profiles for the Jugendwettbewerb Informatik (jwinf.de) and its robot and turtle blocks.
- **MakeCode** — `makecode()`, the micro:bit and Calliope mini editors' blocks in all 36 of their languages.
- **NEPO (Open Roberta)** — `nepo()`, the Calliope mini and micro:bit blocks of Open Roberta Lab.

All four share one text notation: a block per line, `(...)` for a value, `[... v]` for a dropdown, `<...>` for a condition, indentation and an end marker for the body of a loop. Typst hands the text to a bundled WASM plugin, the plugin parses it and returns SVG, Typst embeds the SVG. Nothing has to be installed beyond the package.

CORE DESIGN

- Fully text-based: write blocks as plain text, get rendered blocks. The same notation for every editor.
- Each editor's own geometry, palette and vocabulary — jwinf's classic Blockly measured on jwinf.de, MakeCode's zelos renderer as the editors run it.
- Localized: Scratch's official translations, Blockly's message files, the MakeCode editors' own strings.
- Themes for print (`print`, `grayscale`, `high-contrast`) and the document's own category colours over any palette.
- Line numbers and line labels for worksheets that talk about single lines.

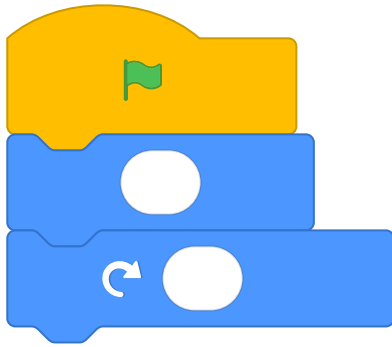
BREAKING CHANGE SINCE 0.2.0

The old pre-0.2.0 native-Typst renderer syntax is **removed**. From 0.2.0 onward, blockst uses only the text-to-WASM pipeline. Documents that still rely on the previous syntax must be migrated.

1.2 Quick start

```
#import "@preview/blockst:0.4.0": scratch, blockly, makecode, nepo

#scratch("
when green flag clicked
move (10) steps
turn cw (15) degrees
")
```



The same document can hold the other editors' blocks. A jwinf task:

```
#blockly("
Roboter-Programm
wiederhole (4) mal:
  gehe nach rechts
  falls <auf Kiste>
    hebe Murmel auf ::aktionen
  ende
ende
", profile: "jwinf")
```

A micro:bit program:

```
#makecode("
beim Start
  zeige Symbol [Herz v]
ende
wenn Knopf [A v] geklickt
  zeige Zahl ((1) + (2))
  pausiere (ms) (100)
ende
")
```

1.3 Which function for which editor

Editor	Function	Profiles	Default language	Code fences
Scratch 3	scratch()	—	"en", 26 languages	scratch via raw-scratch()
Blockly, jwinf	blockly()	blockly (default), blockly-klassisch, jwinf, jwinf-turtle	"de", 24 languages	blockly, jwinf, jwinf-turtle via raw-blockly()
MakeCode	makecode()	makecode (micro:bit, default), makecode-calliope	"de", 36 languages	makecode, microbit, calliope via raw-makecode()
Open Roberta	nepo()	platforms calliope (default), calliopev3, microbit	"de"	nepo via raw-nepo()

Every function takes the same rendering options — theme, scale, font, line numbers, colours — and all of them read the defaults set with `set-blockst()`. The chapters [Scratch](#), [Blockly](#) and [jwinf](#), [MakeCode](#) and [NEPO](#) describe what is particular to each editor.

1.4 Package information

- **Version:** 0.4.0
- **License:** MIT
- **Repository:** github.com/Loewe1000/blockst
- **Compiler requirement:** Typst 0.15.0+
- **Font requirement:** Scratch and Blockly labels are designed for Helvetica Neue. On Linux/Windows install a compatible font (e.g. Nimbus Sans) or set another one with `set-blockst(font: ...)`. MakeCode labels use a monospace face — Menlo, Consolas or DejaVu Sans Mono, whichever is installed.

2 The notation

Blocks are written one per line, the way they read in the editor. Inputs are marked by their brackets, a body is indented and closed by an end marker. The vocabulary is the editor's own in the chosen language — the parser matches against Scratch's translations, Blockly's message files or the MakeCode editors' strings, and a line it does not recognise is drawn as written in a neutral grey (or in the colour of a category you name, see below).

Pattern	Example	Description
Stack block	<code>move (10) steps</code>	A command; blocks on consecutive lines snap together
Hat block	<code>when green flag clicked</code>	An event; starts a new script
Reporter	<code>(x position)</code>	A value with round ends
Boolean	<code><mouse down?></code>	A condition with pointed ends
Input: number	<code>(10)</code>	Numeric input field
Input: string	<code>[hello]</code>	String input field
Input: dropdown	<code>[random position v]</code>	Dropdown selector — the <code>v</code> marks it
Empty input	<code>()</code> , <code>[]</code> , <code><></code>	An empty slot
C-block	<code>repeat (4) ... end</code>	A block with a body: indent the body, close it with the end marker
C-block with else	<code>if <...> then ... else ... end</code>	A second body after the else marker
Icon	<code>@greenFlag</code> , <code>@turnRight</code> , <code>@turnLeft</code>	The flag and the turn arrows
Line label	<code>move (10) steps #step</code>	Names the line, see Labels and line numbers

What differs between the editors is small and follows the editor:

- **End and else markers** follow the language: `end`/`else` in English, `ende`/`sonst` in German Blockly, `ende`/`ansonsten` in German MakeCode, `fin`/`sinon` in French. `end`, `ende` and `else` are understood in every Blockly and MakeCode language.
- **Booleans.** Scratch and MakeCode draw a condition as a hexagon, so `<...>` and `(...)` are different shapes. Blockly has no hexagonal boolean: there `<...>` equals `(...)`.
- **Unknown labels.** Scratch names a category with a prefix, `@motion` free text (see [@category](#)). Blockly and MakeCode use a suffix, `hebe Marmel auf ::aktionen` — on jwinf the normal case, because the same block reads differently from task to task.
- **Units.** MakeCode shows some units in parentheses; they are typed like a value and drawn as the label: `pausiere (ms) (100)`, `Temperatur (°C)`.
- **Editors.** MakeCode's LED matrix and melody editor have a notation of their own, see [MakeCode](#).

3 Options for every editor

`scratch()`, `blockly()`, `makecode()` and `nepo()` accept the same rendering options. Given on the call, they apply to that block group; set with `set-blockst()`, they become the document's defaults; wrapped in `blockst[...]`, they apply to everything inside.

3.1 `set-blockst()` — Global Defaults

```
#set-blockst(
  theme: none,           // "normal", "high-contrast", "print", "grayscale"
  profile: none,         // default profile for blockly() and makecode()
  colors: none,          // (category: colour, ...) over any palette
  scale: none,           // 80%, 0.8 – overall size
  inset-scale: none,     // 60% thin, 125% thick blocks; text size unchanged
  stroke-width: none,
  font: none,            // "Nimbus Sans"
  language: none,        // default language for every editor
  line-numbers: none,
  line-number-start: none,
  line-number-first-block: none,
  line-number-gutter: none,
)
```

All parameters are optional. Only provided values override the current defaults; a later `set-blockst()` changes only what it names.

3.2 `blockst()` — Group Override Container

```
#blockst(theme: "high-contrast", scale: 80%)[
  #scratch("when green flag clicked\nmove (10) steps")
  #blockly("wiederhole (4) mal:\n  gehe nach rechts\nende", profile: "jwinf")
]
```

All parameters match `set-blockst()` but apply only within the body; `spacing` (default `1.5em`) sets the gap between the blocks inside.

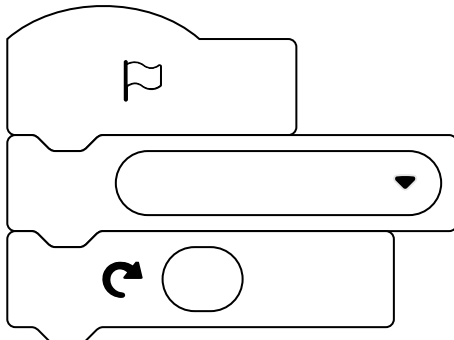
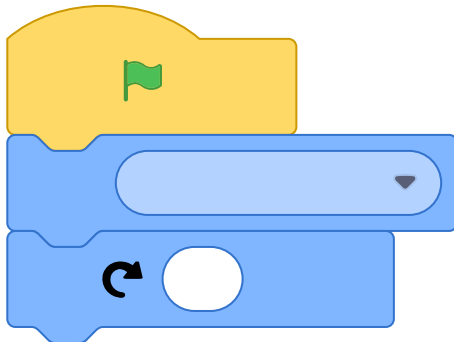
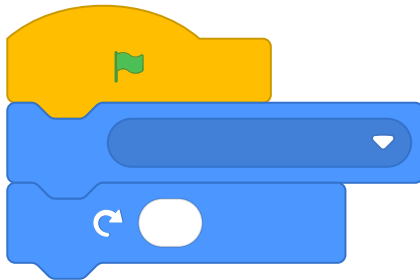
3.3 Themes

Theme	Description
"normal"	Default — the editor's full colours.
"high-contrast"	Lighter, high-contrast variant for accessibility.
"print"	Every block white with a black outline. The lightest on ink, but all categories look alike.
"grayscale"	One distinct grey per category, so a control block still reads differently from an operator on a monochrome page.

The derived themes are computed from the palette in use, so `grayscale` on a `jwinf` profile is `jwinf`'s categories in grey, and a colour set with `colors` carries its own bevel, stroke, high-contrast and grey shades.

```
#let script = "when green flag clicked
go to (random position v)
turn cw (30) degrees"

#blockst(inset-scale: 50%)[#scratch(script)]
#blockst(theme: "high-contrast")[#scratch(script)]
#blockst(theme: "print")[#scratch(script)]
```



3.4 Colours

`colors` lays the document's own category colours over any palette — for the document with `set-blockst()`, for a single block or group on the rendering function or on `blockst()`. A hex string or a Typst colour per category; the category names are the ones the editor uses (`motion`, `looks`, ... for Scratch, `aktionen`, `schleifen`, `logik`, ... for jwinf, `basic`, `input`, `loops`, ... for MakeCode).

```
#set-blockst(colors: (logik: "#73cc47"))
#blockly(programm, profile: "jwinf", colors: (aktionen: "#cc7347", schleifen:
rgb("#5ba55b")))
```

jwinf colours its categories per task family; `colors` is the way to match a task that deviates from both jwinf profiles.

3.5 Fonts

```
#set-blockst(font: "Nimbus Sans")
```

Scratch and Blockly labels default to Helvetica Neue, MakeCode labels to Menlo, Consolas or DejaVu Sans Mono. Set the font here rather than with `set_text(font: ...)`: the renderer measures each label and writes the font into the SVG, so a document-level `set_text` changes what is drawn but not what was measured, and the labels overlap.

3.6 Scale and inset scale

`scale` multiplies the whole drawing (80%, 0.8). `inset-scale` changes the thickness of the blocks without touching the text: 60% gives compact, 125% generous blocks.

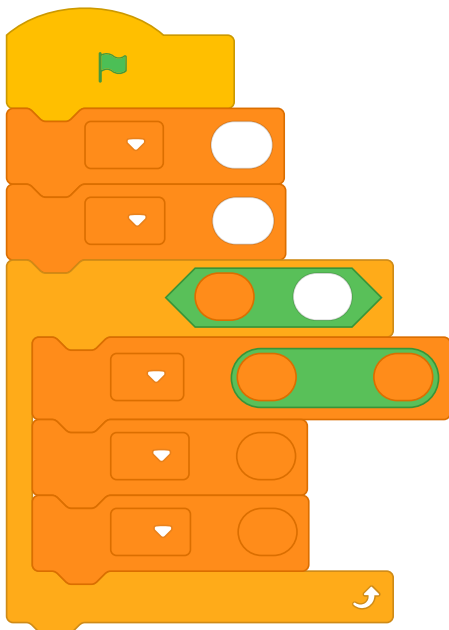
```
#set-blockst(scale: 80%)
#set-blockst(inset-scale: 60%) // compact blocks
```

3.7 Line numbers

```
#set-blockst(line-numbers: true, line-number-start: 1, line-number-gutter: 24)
```

`line-number-start` is the first number, `line-number-gutter` the width of the gutter in points. Numbering continues from one block group to the next; `line-number-first-block` says with which group it starts. The three can also be given as one dictionary, `line-numbering: (enabled: true, start: 1, first-block: 1)`.

Euclidean Algorithm (gcd label walkthrough)



Explanation of labeled lines

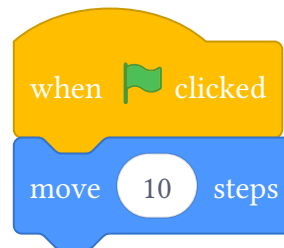
- **Line 4:** Loop condition `b != 0` controls termination.
- **Line 5:** Core rule: `r = a mod b`.
- **Line 7:** State update that advances to the next pair.

4 Scratch

4.1 scratch() — Render Scratch Blocks

Parses Scratch text and renders the blocks the way Scratch 3 draws them.

```
#scratch("when green flag clicked\nmove (10) steps")
```



```
#scratch(
  text,
  language: "en",
  theme: auto,
  scale: auto,
  font: auto,
  colors: auto,
  line-numbers: auto,
  line-number-start: auto,
  line-number-gutter: auto,
  inset-scale: auto,
)
```

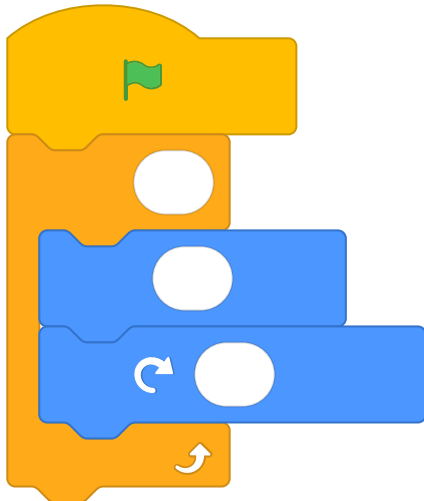
`text` is the Scratch block text in the chosen language; the other parameters are the [options every editor shares](#), `auto` meaning the document's default. The parser knows the full Scratch 3 vocabulary in all 26 locales, including the pen extension.

4.2 raw-scratch — Code fences

The `raw-scratch()` show rule renders `scratch` code fences as blocks:

```
#show: raw-scratch(language: "en")
```

```
when green flag clicked
repeat (4)
  move (30) steps
  turn cw (90) degrees
end
```

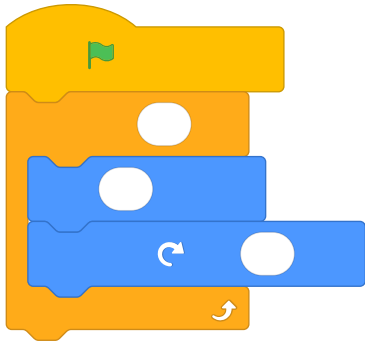


4.3 Languages

Scratch blocks come in **26 languages**, from Scratch's official translations. Set the language on the call or with `set-blockst(language: ...)`; block text must match the chosen locale's vocabulary.

Code	Language	Code	Language
"en"	English	"de"	German
"fr"	French	"es"	Spanish
"it"	Italian	"nl"	Dutch
"pt"	Portuguese	"pl"	Polish
"ru"	Russian	"ja"	Japanese
"ca"	Catalan	"cs"	Czech
"cy"	Welsh	"el"	Greek
"fa"	Persian	"gd"	Scottish Gaelic
"he"	Hebrew	"hi"	Hindi
"hr"	Croatian	"hu"	Hungarian
"id"	Indonesian	"nb"	Norwegian (Bokmål)
"ro"	Romanian	"sl"	Slovenian
"tr"	Turkish	"ar"	Arabic

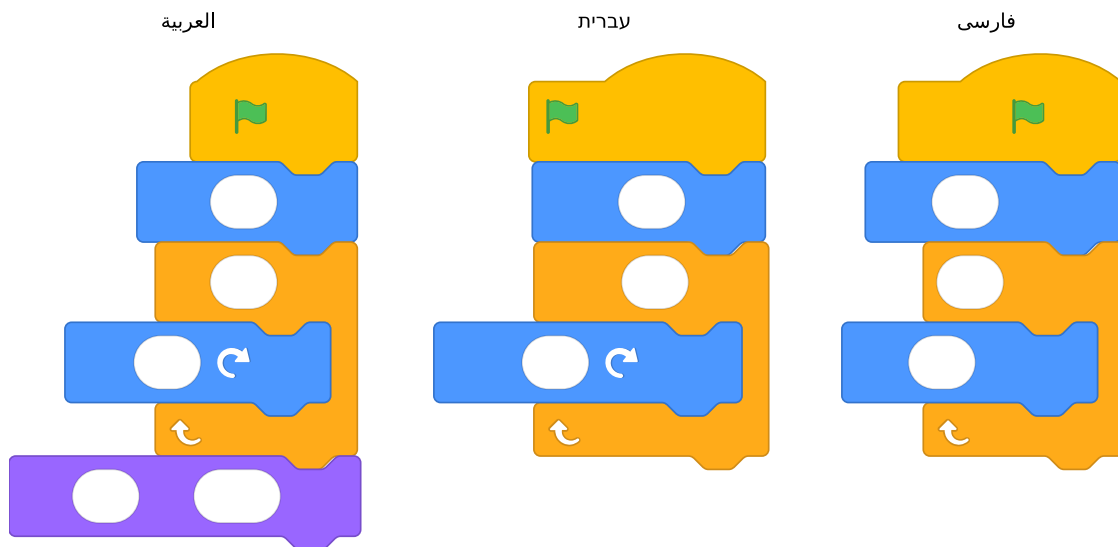
```
#set-blockst(scale: 67.5%)
#scratch("
Wenn die grüne Flagge angeklickt
wiederhole (4) mal
  gehe (30) er Schritt
  drehe dich nach rechts um (90) Grad
end
", language: "de")
```



4.3.1 Right-to-left languages

Arabic ("ar"), Hebrew ("he") and Persian ("fa") render right-to-left. Nothing has to be switched on: each locale declares its own direction, and the renderer mirrors the layout — the notch, the hat dome, the C-block mouth, the loop arrow, the pen badge, the `define` hat and the order of the labels all move to the reading edge.

```
#scratch("
عند نقر @greenFlag
تحرك (10) خطوة
كرّر (4) مرة
درجة (90) @turnRight استدر
نهاية
", language: "ar")
```



WHAT IS MIRRORED, AND WHAT IS NOT

Layout is mirrored; meaning is not. The loop arrow points back to the top of the loop, which is a claim about the drawing, so it flips with the drawing. `@turnRight` and `@turnLeft` are never mirrored — their direction is what the sprite is being told to do, and a mirrored `@turnRight` would tell the reader to turn the other way.

Arabic short vowels are optional and their order is not canonical, so blocks match whether or not you type the harakat: كَرّر and كَرّر both find the repeat block.

4.4 @category — Quick Color Defaults

A `@category` prefix forces a block's category colour, even without matching the full localized syntax.

```
@motion           // → default motion block
@motion free text // → unrecognized block in motion color
```

Token	Normalized	Default Block
@motion	motion	move (10) steps
@looks	looks	say [Hello!]
@sound	sound	play sound [pop v]
@events	events	when green flag clicked
@control	control	repeat (10)
@sensing	sensing	ask [What's your name?] and wait
@operators	operators	(() + ())
@variable	variables	set [var v] to (0)
@list	lists	add (thing) to [list v]
@pen	pen	clear
@event	events	(alias for events)
@operator	operators	(alias for operators)

When `@category` is followed by text that matches a known block in that category, the actual block is rendered; otherwise the fallback is an unrecognized block in the forced colour.

```
@list add (12) to [my list v] // → DATA_ADDTOLIST
@variable change [score v] by (1) // → DATA_CHANGEVARIABLEBY
```

4.5 scratch-parse() — Parse to AST

Parses Scratch text to an abstract syntax tree for programmatic use — a nested structure of blocks, inputs and bodies.

```
#scratch-parse(text, language: "en")
```

The complete list of Scratch blocks, rendered live, is the [Scratch block catalog](#) at the end of this manual. Running Scratch programs as turtle graphics and importing `.sb3` files have chapters of their own.

5 Blockly and jwinf

5.1 blockly() — Render Blockly Blocks

`blockly()` renders Blockly blocks from the shared notation, drawn with Blockly's shapes — notch, puzzle tab, boxed fields, the mutator gear on an if block, the warning sign on a loop-control block outside a loop.

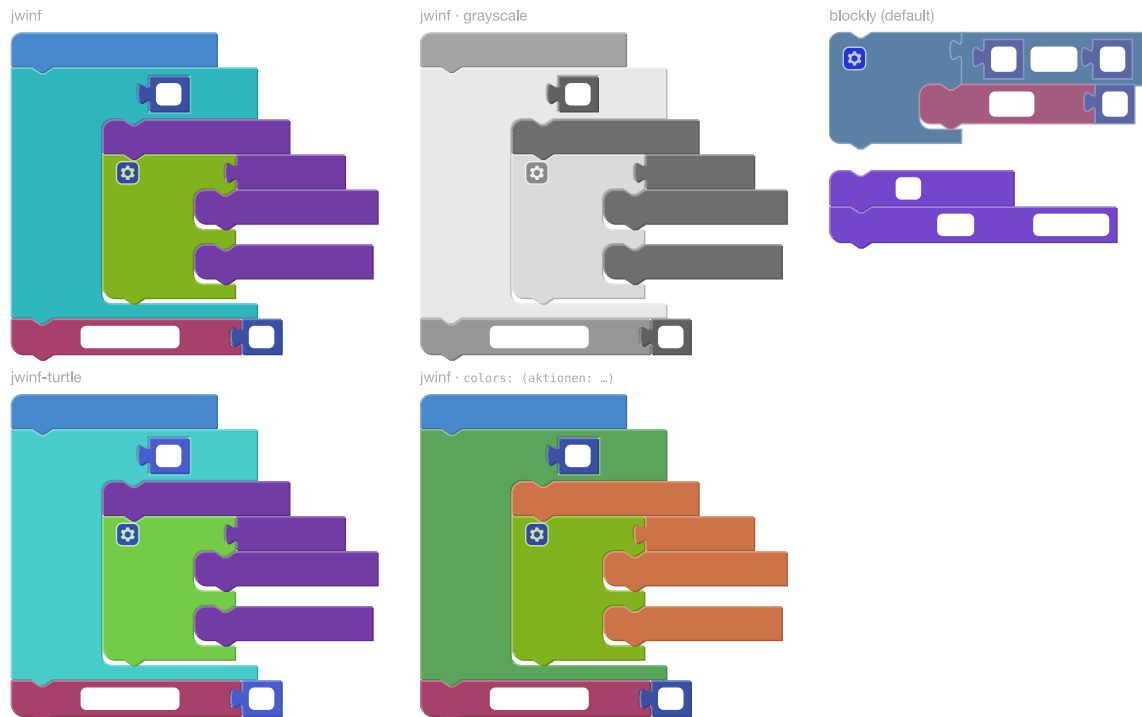
```
#blockly(
  text,
  profile: auto,      // "blockly", "blockly-klassisch", "jwinf", "jwinf-turtle"
  language: auto,     // "de" by default
  theme: auto,
  scale: auto,
  font: auto,
  colors: auto,
  line-numbers: auto,
  line-number-start: auto,
  line-number-gutter: auto,
  inset-scale: auto,
)
```

5.2 Profiles

A profile chooses look and vocabulary. `set-blockst(profile: ...)` sets the default for `blockly()`.

Profile	Look	Vocabulary
"blockly" (default)	today's flat Blockly (thrasos)	Blockly's standard blocks
"blockly-klassisch"	Blockly before 2019, with the bevel highlight	same
"jwinf"	jwinf.de — the classic geometry measured on the site, the palette of the robot training tasks	robot and turtle world blocks, plus the standard blocks in the old wording
"jwinf-turtle"	the same, with the colours of the Freie Turtle-Umgebung	same

```
#blockly("
  Roboter-Programm
  wiederhole (4) mal:
    gehe nach rechts
    falls <auf Kiste>
      hebe Murmel auf ::aktionen
    ende
  ende
", profile: "jwinf")
```



5.3 Writing jwinf tasks

On jwinf the same block reads differently from task to task — „hebe Murmel auf,, „nimm Fisch“, „Holzstapel einsammeln,, — so there is no fixed vocabulary to match against. Write the label as it appears and name the category with a `::` suffix: `aktionen`, `schildkroete`, `sensoren`, `schleifen`, `logik`, `mathe`, `variablen`, `funktionen`, `text`, `listen`, `ausgeben`, `einlesen`. Blocks the profile knows — loops, conditions, variables, the robot and turtle commands — are recognised without a suffix.

`ende` closes a C-block, `sonst` opens its else branch, and `<...>` equals `(...)`: Blockly has no hexagonal boolean. Where a task's colours deviate from both profiles, `colors` lays the document's own over the palette (see [Colours](#)).

5.4 Languages

The standard blocks come in 24 languages, from Blockly's own message files: `language: "de"` (the default), `"en"`, `"fr"` (`répéter (10) fois`), `"ja"` (`((10) □□□□□)`), and `ar`, `ca`, `cs`, `el`, `es`, `fa`, `he`, `hi`, `hr`, `hu`, `id`, `it`, `nb`, `nl`, `pl`, `pt`, `ro`, `ru`, `sl`, `tr`. The jwinf world blocks exist in German only. Each language closes a C-block with the marker its Scratch locale uses (`ende`, `end`, `fin`, ...) and opens the else branch with Blockly's word (`sonst`, `else`, `sinon`, ...); `ende`, `end` and `else` work in every language.

5.5 Code fences and parsing

`raw-blockly()` renders `blockly` fences with the default profile, `jwinf` fences with the jwinf profile and `jwinf-turtle` fences with the turtle colours, whatever the arguments say. `blockly-parse()` returns the AST.

```
#show: raw-blockly()
#blockly-parse(text, language: "de", profile: "blockly")
```

Themes apply as for Scratch, with `grayscale` derived from the profile's palette.

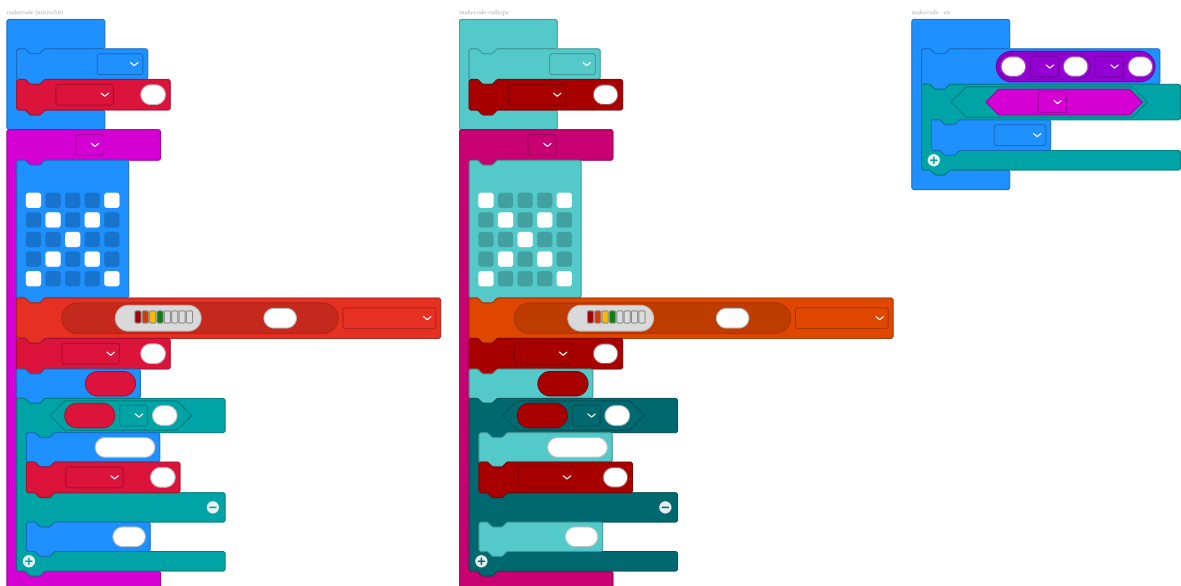
6 MakeCode

6.1 makecode() — Render MakeCode Blocks

`makecode()` renders the blocks of the MakeCode editors for the micro:bit and the Calliope mini. The look is Blockly's zelos renderer as the editors run it — pills and hexagons as tall as their block, white literal pills, the – and + buttons of an if block, MakeCode's monospace labels — and the block texts are the editors' own.

```
#makecode(
  text,
  profile: auto,      // "makecode" (micro:bit), "makecode-calliope"
  language: auto,     // "de" by default
  theme: auto,
  scale: auto,
  font: auto,
  colors: auto,
  line-numbers: auto,
  line-number-start: auto,
  line-number-gutter: auto,
  inset-scale: auto,
)
```

```
#makecode("
beim Start
  zeige Symbol [Herz v]
ende
wenn Knopf [A v] geklickt
  zeige Zahl ((1) + (2))
  pausiere (ms) (100)
ende
")
```



6.2 Profiles

"makecode" (the default) is the micro:bit editor, "makecode-calliope" the Calliope mini's: the same blocks plus the Calliope's own (motors, the RGB LED), in the Calliope palette. `set-blockst(profile: "makecode-calliope")` sets the default for `makecode()`; `colors:` overrides single categories.

6.3 Writing MakeCode programs

- **Units.** A unit the editor shows in parentheses is typed like a value and drawn as the label: `pausiere (ms) (100)`, `Temperatur (°C)`.
- **Operators** take the editor's glyph or a spelling: `×` or `*`, `÷` or `/`, `≥` or `>=`, `≤` or `<=`, `≠` or `!=`.
- **LED matrix.** 25 cells, `#` lit and `.` dark, in any grouping: `zeige LEDs [#...#|.#.#.|..#..|.#.#|#...#]`.
- **Melody editor.** Eight notes or rests: `spiele (Melodie [C D E F - - -] mit Tempo (120) (bpm)) [bis zum Ende v]`.
- **Else.** `ansonsten / else` opens the else branch, which gets the editor's – and + buttons.
- **Unknown labels** are drawn as written with the category from a `::kategorie` suffix: `basic`, `input`, `music`, `led`, `radio`, `loops`, `logic`, `variables`, `math`, `functions`, `arrays`, `text`, `game`, `images`, `pins`, `serial`, `control`.

6.4 Languages

The block texts are the editors' own in every language the editors offer: German (`language: "de"`, the default) and English read off the running editors, the other 34 – `"fr"`, `"es"`, `"ja"`, `"zh-cn"`, ... – from the translation service the editors load at run time (approved strings only; an untranslated block keeps its English text, as in the editor). A bare code picks the regional variant the editor ships (`"es"` → `es-ES`, `"pt"` → `pt-BR`). The end and else markers follow the language (`fin / sinon`, `□□□ / □□□□□`, ...); `ende`, `end` and `else` work everywhere.

6.5 Code fences and parsing

`raw-makecode()` renders `makecode` and `microbit` fences with the micro:bit profile and `calliope` fences with the Calliope mini profile. `makecode-parse()` returns the AST.

```
#show: raw-makecode()
#makecode-parse(text, language: "de", profile: "makecode")
```

7 NEPO (Open Roberta)

`nepo()` renders the blocks of Open Roberta Lab for the Calliope mini and the micro:bit — a renderer of its own, contributed by [lkoehl](#). Block text prints in Open Roberta's own wording even when the source used a colloquial alias.

```
#nepo(
  code,
  language: "de",
  platform: "calliope", // "calliope", "calliopev3", "microbit"
  theme: auto,
  scale: auto,
  font: auto,
)
```

```
#nepo("
Start
  Zeige Text \"Hallo\"
  Wiederhole unendlich oft
    Schalte RGB LED an (#ff0000)
  Ende
")
```



`raw-nepo()` renders `nepo` fences, `nepo-parse()` returns the block tree. The renderer is a prototype: the set of blocks is the beginner set of the three platforms, and `examples/nepo` in the repository holds the programme collection and the comparison sheet against Open Roberta's own drawings.

8 Labels and line numbers

A line that ends in `#name` is labelled. Labels are collected from every rendered block group — Scratch, Blockly, MakeCode — and can be queried later, so a worksheet can say „the loop in line 3“ and be right after the program changes.

8.1 `blockst-labels()` — Query Labels

```
#blockst-labels("loop") // → line number or "NaN"  
#blockst-labels()       // → full label → line dictionary
```

8.2 `scratch-labels()` — Extract Labels

Parses Scratch text and returns a dictionary mapping label names to line numbers, without rendering.

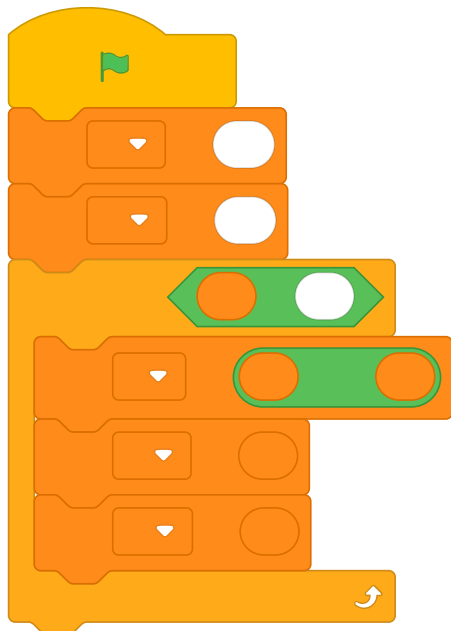
```
#let labels = scratch-labels("  
repeat (4) #loop  
  move (20) steps #step  
  turn cw (90) degrees  
end  
")  
// labels = {"loop": 1, "step": 2}
```

8.3 `blockst-register-labels()` — Pre-register Labels

Registers labels globally without rendering blocks — for labels that are needed before the first block output.

```
#blockst-register-labels("  
repeat (4) #loop  
  move (20) steps #step  
end  
")
```

Euclidean Algorithm (gcd label walkthrough)



Explanation of labeled lines

- **Line 4:** Loop condition $b \neq 0$ controls termination.
- **Line 5:** Core rule: $r = a \bmod b$.
- **Line 7:** State update that advances to the next pair.

9 Running Scratch programs

blockst includes an execution engine that runs Scratch programs visually — for demonstrating program flow and pen drawing. It runs Scratch text only.

9.1 scratch-run Module

```
#import "@preview/blockst:0.4.0": scratch-run, set-scratch-run

#scratch-run.stage("...", scale: 2)
#scratch-run.grid("...", grid: true)
```

9.1.1 scratch-run.stage() — Stage Canvas

Renders pen drawing on a stage canvas, like the Scratch stage.

```
#scratch-run.stage(
  program,
  language: "en",      // locale for block text
  size: auto,          // (width, height) in pixels (default 480×360)
  scale: auto,         // drawing scale multiplier
  start: auto,         // (x: 0, y: 0, angle: 90)
  pen: auto,           // (down: false, color: ..., size: ...)
  background: auto,    // background color (e.g. white, black)
  cursor: auto,        // show/hide turtle cursor (default: true)
  border: auto,        // show/hide stage border (default: true)
)
```

9.1.2 scratch-run.grid() — Coordinate Grid

Renders the drawing on a Cartesian coordinate grid with optional axes and grid lines.

```
#scratch-run.grid(
  program,
  language: "en",      // locale for block text
  x: auto,             // view bounds (tuple, e.g. (-10, 10))
  y: auto,             // view bounds (tuple)
  step: auto,          // grid step size
  scale: auto,         // drawing scale multiplier
  start: auto,         // (x: 0, y: 0, angle: 90)
  pen: auto,           // (down: false, color: ..., size: ...)
  background: auto,    // background color
  axes: auto,          // show axis lines (default: false)
  grid: auto,          // show grid lines (default: false)
  grid-style: auto,    // grid line stroke (default: 0.5pt + gray)
  cursor: auto,        // show/hide turtle cursor (default: true)
  fit: auto,           // auto-fit view to drawing bounds
)
```

9.1.3 set-scratch-run() — Run Global Defaults

```
#set-scratch-run(
  scale: none,         // default scale for all runs
  start: none,         // (x: 0, y: 0, angle: 90)
  pen: none,           // (down: false, color: ..., size: ...)
  background: none,    // default background color
  cursor: none,        // show/hide turtle cursor (default: true)
```

```

stage: none,          // (size: (480, 360), border: true)
grid: none,           // (visible: false, axes: false, step: auto, style: auto)
)

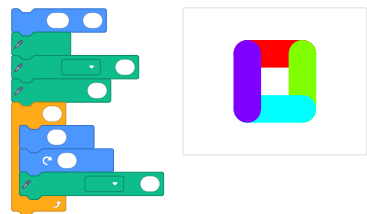
```

9.1.4 Supported Execution Commands

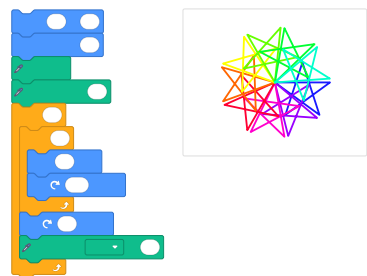
Category	Commands
Motion	move, turn-right, turn-left, set-direction, go-to, set-x, set-y, change-x, change-y
Pen	pen-down, pen-up, set-pen-color, set-pen-size, change-pen-size, erase-all, stamp, set-pen-param, change-pen-param
Variables	set-variable, change-variable, variable
Operators	plus, minus, multiply, divide, modulo, random, round, greater, less, equals, op-and, op-or, op-not
Control	repeat, repeat-until, if-then, if-else, wait
Looks	say, think
Shapes	square, triangle, circle, star, spiral
German	gehe, drehe-rechts, drehe-links, setze-richtung, gehe-zu, stift-ein, stift-aus, setze-stiftfarbe-auf, setze-stiftdicke, setze-variable, aendere-variable, groesser, kleiner, gleich, und, oder, nicht, mal, geteilt, zufallszahl

9.1.5 Complete Example

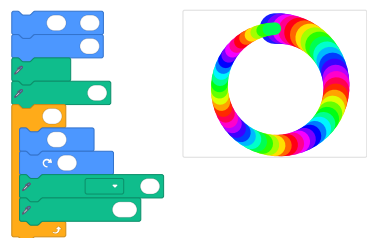
1) Hue Square



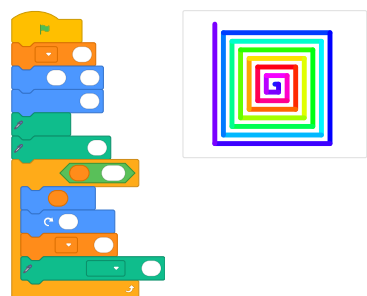
2) Star Rosette (Nested repeat)



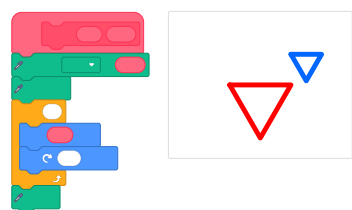
3) Colored Spiral (Hue + pen size)



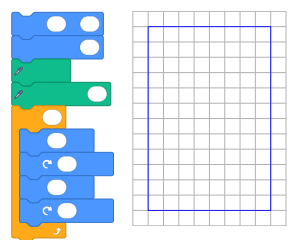
4) Conditional Loop



5) Custom Block: Triangle Pattern



6) Grid Preview (Axes + fixed bounds)



```
#let square-program = "  
go to x: (-45) y: (45)  
pen down  
set pen [color v] to (0)  
set pen size to (45)  
repeat (4)  
  move (90) steps  
  turn cw (90) degrees  
  change pen [color v] by (25)  
end"  
  
#set-scratch-run(  
  stage: (size: (300, 240)),  
  start: (x: 0, y: 0, angle: 90),  
)  
  
#grid(  
  columns: (auto, auto),  
  gutter: 6mm,  
  [#scratch(square-program)],  
  [#scratch-run.stage(square-program, scale: 2)],  
)
```

10 Importing Scratch projects (SB3)

blockst can read real Scratch 3 project files (`.sb3`) and extract scripts, variables, lists, images, and screen previews.

10.1 Basic Workflow

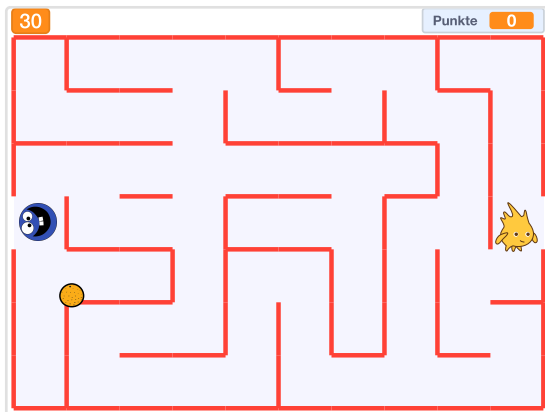
```
#let project = read("my-project.sb3", encoding: none)

#sb3.render-sb3-scripts(project, language: "en", target: "Sprite1")
```

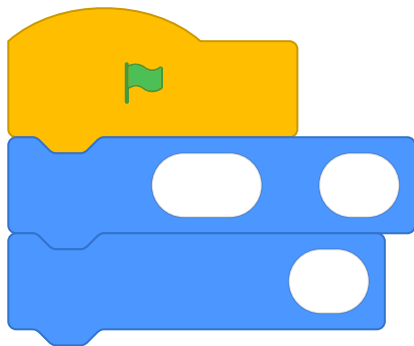
The `.sb3` file must be read as raw bytes (`encoding: none`).

10.2 render-sb3-scripts()

```
#sb3.render-sb3-scripts(
  sb3-bytes,           // raw .sb3 file bytes (read with encoding: none)
  script-number: auto, // global script index (1-based)
  target-script-number: auto, // script index within selected target
  target: auto,        // filter by target name ("Stage", sprite name, or auto for
all)
  sb3-plugin: auto,    // WASM plugin path (auto = bundled plugin)
  language: "en",      // locale for block text
  show-headers: auto,  // show target name header
  header-gap: 1.5mm,   // spacing between target name and scripts
  script-gap: 3mm,     // spacing between individual scripts
)
```



2. Sprite: Pacman - Script 2



10.3 render-sb3-variables()

```
#sb3.render-sb3-variables(
  sb3-bytes,           // raw .sb3 file bytes
  target: auto,        // filter by target name
  target-variable-name: auto, // filter by variable name
  target-variable-number: auto, // variable index within target (1-based)
  sb3-plugin: auto,    // WASM plugin path
  language: "en",      // locale for display text
  show-target-headers: auto, // show target name header
  target-gap: 2mm,     // spacing between targets
  item-gap: 0.8mm,     // spacing between variable items
)
```

10.4 render-sb3-lists()

```
#sb3.render-sb3-lists(
  sb3-bytes,           // raw .sb3 file bytes
  target: auto,        // filter by target name
  target-list-name: auto, // filter by list name
  target-list-number: auto, // list index within target (1-based)
  sb3-plugin: auto,    // WASM plugin path
  language: "en",      // locale for display text
  show-target-headers: auto, // show target name header
  target-gap: 2mm,     // spacing between targets
  item-gap: 0.8mm,     // spacing between list items
)
```

10.5 Standalone Monitors

10.5.1 list-monitor()

```
#list-monitor(
  name: "List",          // display name shown in header
  items: (),             // array of values to display
  width: 5.2cm,          // monitor width
  height: auto,          // auto-grows to fit content
  length-label: auto,    // show length indicator (e.g. "length: 3")
)
```

10.5.2 variable-monitor()

```
#variable-monitor(
  name: "Variable",     // display name shown in header
  value: 0,              // current value to display
)
```

Counter 30

Punkte 0

Highscore	
1	Thomas
2	Alex
3	Lukas
+ Length: 3 =	

10.6 screen-preview()

```
#sb3.sb3-screen-preview(
  sb3-bytes,            // raw .sb3 file bytes
  width: 480,           // stage rendering width in pixels
  height: 360,          // stage rendering height in pixels
  unit: 1,              // size multiplier (2 = double size)
  background: none,     // override background color
  show-border: true,    // show stage border
  show-backdrop: true,  // show costume/backdrop image
  monitor-scale: 1.5,   // scale factor for variable/list overlays
  language: auto,       // locale for monitor text
)
```

Renders a static Scratch stage preview with sprites, backdrop, and monitors.

10.7 Image Helpers

```
// List all image assets in the project
#sb3.sb3-image-assets-catalog(sb3-bytes, target: auto)

// Render a single image asset
#sb3.sb3-image(
  sb3-bytes,            // raw .sb3 file bytes
```

```
target: auto,          // filter by sprite/stage name
image-number: auto,    // global image index (1-based)
target-image-number: auto, // image index within target (1-based)
image-name: auto,      // filter by image name (e.g. "costume1")
width: auto,           // output width (auto = original size)
height: auto,          // output height
)
```

10.8 Catalog Helpers

```
// Grouped script metadata (targets, scripts, blocks count)
#sb3.sb3-scripts-catalog(sb3-bytes)

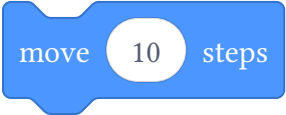
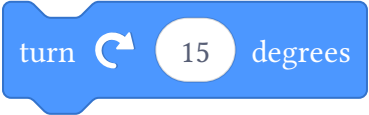
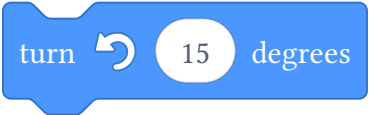
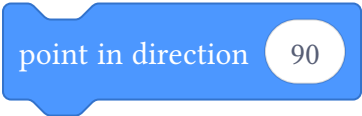
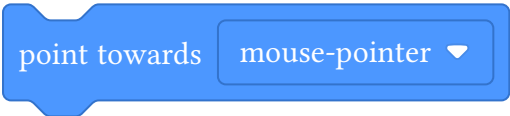
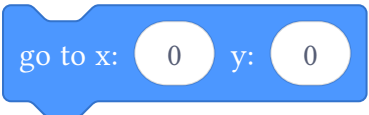
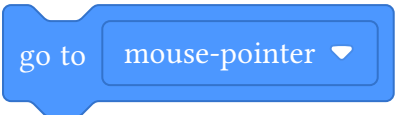
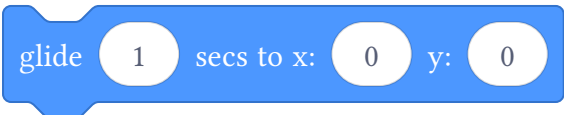
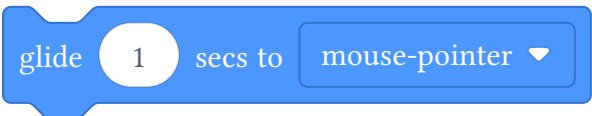
// Target states (variables, lists, and sprite properties)
#sb3.sb3-state-catalog(sb3-bytes)

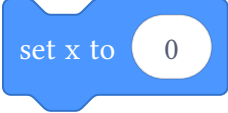
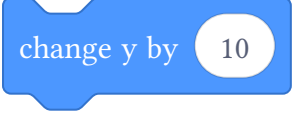
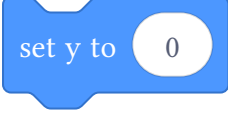
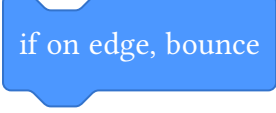
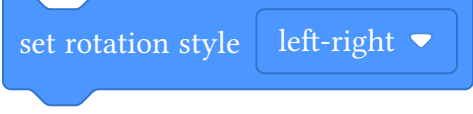
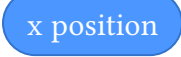
// Convert a specific SB3 script to Scratch text
#sb3.sb3-bytes-to-scratch-text(
  sb3-bytes,          // raw .sb3 file bytes
  script-number: auto, // global script index (1-based)
  language: "en",     // locale for output text
)
```

11 Scratch block catalog

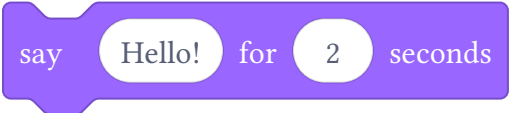
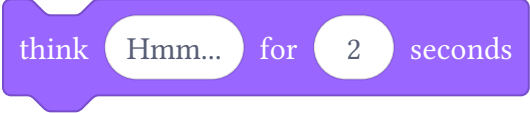
Every block of each Scratch 3 category, rendered live, next to the text that produces it. Blockly's and MakeCode's vocabularies are the editors' own and too large to list here; the generator reports in the repository (`scripts/scratchblocks-wasm/data/dialects/REPORT.md`) hold them per language.

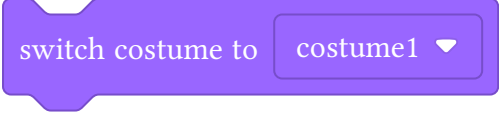
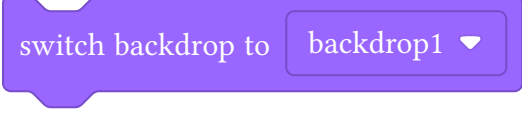
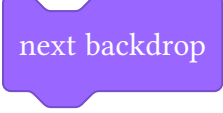
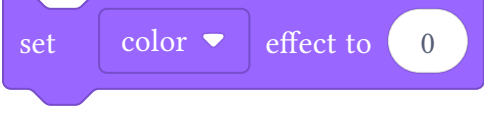
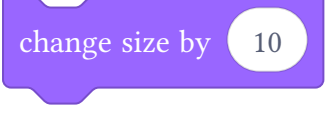
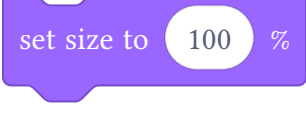
11.1 Motion

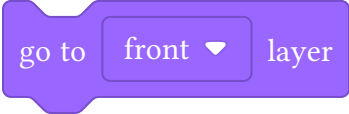
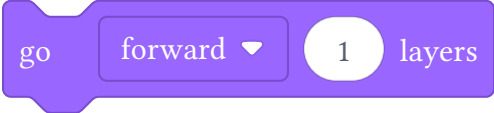
Block	Code
	<code>move (10) steps</code>
	<code>turn cw (15) degrees</code>
	<code>turn ccw (15) degrees</code>
	<code>point in direction (90)</code>
	<code>point towards [mouse-pointer v]</code>
	<code>go to x: (0) y: (0)</code>
	<code>go to [mouse-pointer v]</code>
	<code>glide (1) secs to x: (0) y: (0)</code>
	<code>glide (1) secs to [mouse-pointer v]</code>

Block	Code
	<code>change x by (10)</code>
	<code>set x to (0)</code>
	<code>change y by (10)</code>
	<code>set y to (0)</code>
	<code>if on edge, bounce</code>
	<code>set rotation style [left-right v]</code>
	<code>(x position)</code>
	<code>(y position)</code>
	<code>(direction)</code>

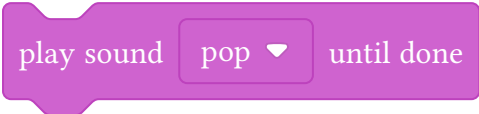
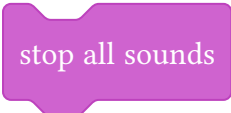
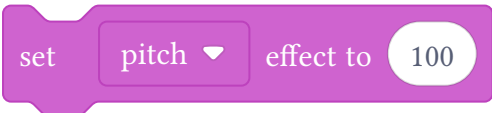
11.2 Looks

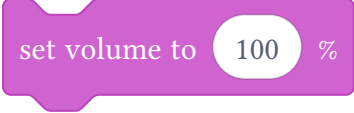
Block	Code
	<code>say [Hello!] for (2) seconds</code>
	<code>say [Hello!]</code>
	<code>think [Hmm...] for (2) seconds</code>

Block	Code
	<code>think [Hmm...]</code>
	<code>show</code>
	<code>hide</code>
	<code>switch costume to [costume1 v]</code>
	<code>next costume</code>
	<code>switch backdrop to [backdrop1 v]</code>
	<code>next backdrop</code>
	<code>change [color v] effect by (25)</code>
	<code>set [color v] effect to (0)</code>
	<code>clear graphic effects</code>
	<code>change size by (10)</code>
	<code>set size to (100) %</code>

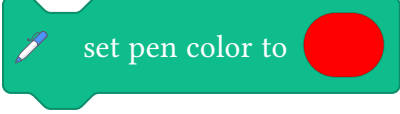
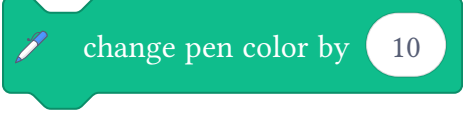
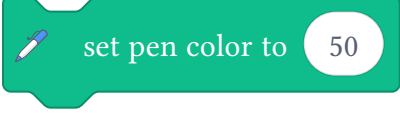
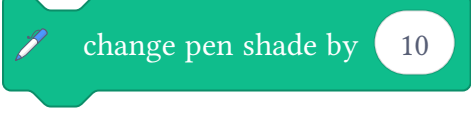
Block	Code
	<code>go to [front v] layer</code>
	<code>go [forward v] (1) layers</code>
	<code>(costume [number v])</code>
	<code>(backdrop [number v])</code>
	<code>(size)</code>

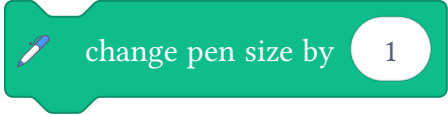
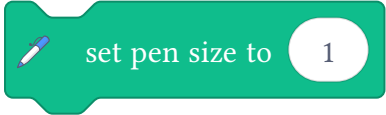
11.3 Sound

Block	Code
	<code>play sound [pop v] until done</code>
	<code>start sound [pop v]</code>
	<code>stop all sounds</code>
	<code>change [pitch v] effect by (10)</code>
	<code>set [pitch v] effect to (100)</code>
	<code>clear sound effects</code>

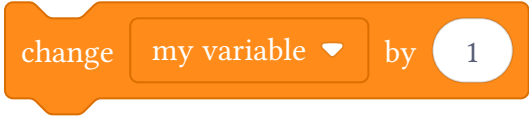
Block	Code
	<code>change volume by (-10)</code>
	<code>set volume to (100) %</code>
	<code>(volume)</code>

11.4 Pen

Block	Code
	<code>erase all</code>
	<code>stamp</code>
	<code>pen down</code>
	<code>pen up</code>
	<code>set pen color to [#ff0000]</code>
	<code>change pen color by (10)</code>
	<code>set pen color to (50)</code>
	<code>change pen shade by (10)</code>

Block	Code
	<code>set pen shade to (50)</code>
	<code>change pen size by (1)</code>
	<code>set pen size to (1)</code>

11.5 Variables

Block	Code
	<code>set [my variable v] to (0)</code>
	<code>change [my variable v] by (1)</code>
	<code>show variable [my variable v]</code>
	<code>hide variable [my variable v]</code>
	<code>(my variable)</code>

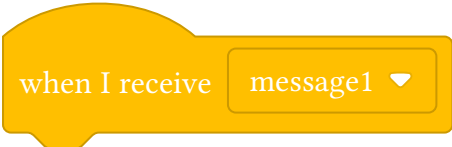
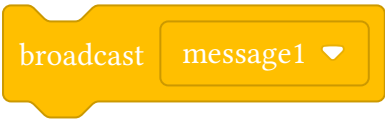
11.6 Lists

Block	Code
	<code>add [thing] to [my list v]</code>
	<code>delete (1) of [my list v]</code>

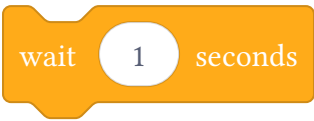
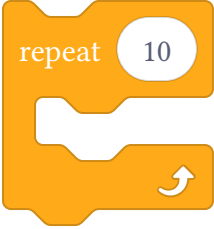
Block	Code
	<code>delete all of [my list v]</code>
	<code>insert [thing] at (1) of [my list v]</code>
	<code>replace item (1) of [my list v] with [thing]</code>
	<code>(item (1) of [my list v])</code>
	<code>(item # of [thing] in [my list v])</code>
	<code>(length of [my list v])</code>
	<code><[my list v] contains [thing]?></code>
	<code>show list [my list v]</code>
	<code>hide list [my list v]</code>

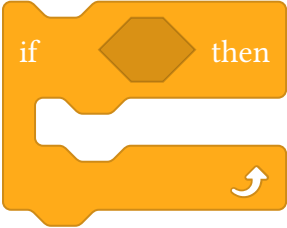
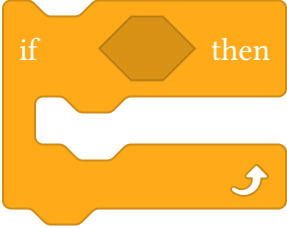
11.7 Events

Block	Code
	<code>when green flag clicked</code>
	<code>when [space v] key pressed</code>

Block	Code
	<code>when this sprite clicked</code>
	<code>when backdrop switches to [backdrop1 v]</code>
	<code>when [loudness v] > (10)</code>
	<code>when I receive [message1 v]</code>
	<code>broadcast [message1 v]</code>
	<code>broadcast [message1 v] and wait</code>

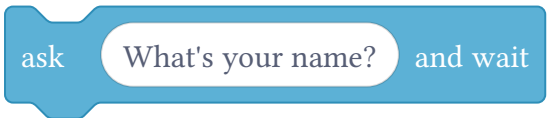
11.8 Control

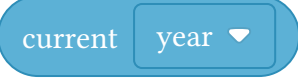
Block	Code
	<code>wait (1) seconds</code>
	<code>repeat (10)</code>
	<code>end</code>

Block	Code
	<code>forever</code>
	<code>end</code>
	<code>if <> then</code>
	<code>end</code>
	<code>if <> then</code>
	<code>else</code>
	<code>end</code>
	<code>wait until <></code>
	<code>repeat until <></code>

Block	Code
	<code>end</code>
	<code>stop [all v]</code>
	<code>when I start as a clone</code>
	<code>create clone of [myself v]</code>
	<code>delete this clone</code>

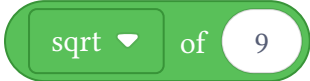
11.9 Sensing

Block	Code
	<code><touching [mouse-pointer v]?></code>
	<code><touching color [#ff0000]?></code>
	<code><color [#ff0000] is touching [#00ff00]?></code>
	<code>(distance to [mouse-pointer v])</code>
	<code>ask [What's your name?] and wait</code>
	<code>(answer)</code>
	<code><key [space v] pressed?></code>
	<code><mouse down?></code>

Block	Code
	<code>(mouse x)</code>
	<code>(mouse y)</code>
	<code>(loudness)</code>
	<code>(timer)</code>
	<code>reset timer</code>
	<code>([x position v] of [Sprite1 v])</code>
	<code>(current [year v])</code>
	<code>(days since 2000)</code>
	<code>(username)</code>

11.10 Operators

Block	Code
	<code>(() + ())</code>
	<code>(() - ())</code>
	<code>(() * ())</code>
	<code>(() / ())</code>
	<code>(pick random (1) to (10))</code>

Block	Code
	<code><() > ()></code>
	<code><() < ()></code>
	<code><() = ()></code>
	<code><> and <></code>
	<code><> or <></code>
	<code>not <></code>
	<code>(join [apple] [banana])</code>
	<code>(letter (1) of [apple])</code>
	<code>(length of [apple])</code>
	<code>([apple] contains [a]?)</code>
	<code>(() mod ())</code>
	<code>(round ())</code>
	<code>([sqrt v] of (9))</code>

12 Contributing

Contributions are welcome: bug reports, missing blocks, parser improvements, rendering polish, docs, and new localizations.

- **Repository:** github.com/Loewe1000/blockst
- **License:** MIT

13 API Reference

The reference is generated from the `///` comments in the source. It covers the public rendering API in `libs/scratch/api.typ`, which `lib.typ` and `package.typ` re-export unchanged.

13.1 blockly

```
blockly(
  text,
  profile: auto,
  language: auto,
  theme: auto,
  scale: auto,
  font: auto,
  line-numbering: auto,
  line-numbers: auto,
  line-number-start: auto,
  line-number-first-block: auto,
  line-number-gutter: auto,
  inset-scale: auto,
  colors: auto,
)
```

Render Blockly blocks from text. Same notation as `scratch()`, drawn with Blockly's shapes — notch, puzzle tab, boxed fields — and the vocabulary, categories and colours of a profile:

- `"blockly"` (the default): today's flat look, Blockly's German wording
- `"blockly-klassisch"`: the pre-2019 look
- `"jwinf"`: jwinf.de — classic geometry, the robot and turtle world blocks, and the palette of the robot training tasks
- `"jwinf-turtle"`: the same with the colours of the Freie Turtle-Umgebung

`colors: (logik: "#73cc47")` overrides single categories of any profile.

Labels that no profile knows are drawn as written, with the category taken from a `::kategorie` suffix — on jwinf the normal case, because the same block reads differently from task to task.

```
#blockly("wiederhole (4) mal:\n gehe nach rechts\nende", profile: "jwinf")
```

Name	Vorgabe
text	—
profile	auto
language	auto
theme	auto
scale	auto
font	auto
line-numbering	auto
line-numbers	auto
line-number-start	auto
line-number-first-block	auto
line-number-gutter	auto
inset-scale	auto

Name	Vorgabe
colors	auto

13.2 blockly-parse

```
blockly-parse(text, language: "de", profile: "blockly")
```

Parse Blockly text to AST (for programmatic use).

Name	Vorgabe
text	–
language	"de"
profile	"blockly"

13.3 blockst

```
blockst(
  theme: auto,
  scale: auto,
  font: auto,
  line-numbering: auto,
  line-numbers: auto,
  line-number-start: auto,
  line-number-first-block: auto,
  line-number-gutter: auto,
  inset-scale: auto,
  language: auto,
  colors: auto,
  spacing: 1.5em,
  body,
)
```

Optional container for grouped blocks with theme/scale override. Only needed when a specific group should differ from global settings.

Name	Vorgabe
theme	auto
scale	auto
font	auto
line-numbering	auto
line-numbers	auto
line-number-start	auto
line-number-first-block	auto
line-number-gutter	auto
inset-scale	auto
language	auto
colors	auto
spacing	1.5em
body	–

13.4 blockst-labels

```
blockst-labels(name)
```

Read globally collected line labels from all previously rendered `scratch()` blocks. With a name: `#blockst-labels("start", default: "?")`. Without a name: returns the full label-to-line dictionary.

13.5 blockst-register-labels

```
blockst-register-labels(text, language: "en")
```

Register labels globally without rendering blocks. Useful when labels are needed before the first `#scratch()` output appears.

Name	Vorgabe
text	–
language	"en"

13.6 makecode

```
makecode(
  text,
  profile: auto,
  language: auto,
  theme: auto,
  scale: auto,
  font: auto,
  line-numbering: auto,
  line-numbers: auto,
  line-number-start: auto,
  line-number-first-block: auto,
  line-number-gutter: auto,
  inset-scale: auto,
  colors: auto,
)
```

Render MakeCode blocks from text — the micro:bit and Calliope mini editors' look: Blockly's `zelos` renderer with MakeCode's monospace labels. Same notation as `scratch()`; `(...)` is a value, `[... v]` a dropdown, `<...>` a boolean, `ende` / `end` closes a C-block and `sonst` / `else` opens the else branch. The profile picks the target: `"makecode"` (micro:bit, the default) or `"makecode-calliope"`; the language is any of the editors' 36 (`"de"` by default, `"en"`, `"fr"`, `"es"`, `"zh-cn"`, ...), with the block texts the editors show in that language. The end marker follows the language (`ende`, `end`, `fin`, ...); `end` and `ende` work in every language.

```
#makecode("beim Start\n zeige Zahl (0)\nende")
```

Name	Vorgabe
text	–
profile	auto
language	auto
theme	auto
scale	auto
font	auto
line-numbering	auto
line-numbers	auto

Name	Vorgabe
line-number-start	auto
line-number-first-block	auto
line-number-gutter	auto
inset-scale	auto
colors	auto

13.7 makecode-parse

```
makecode-parse(text, language: "de", profile: "makecode")
```

Parse MakeCode text to AST (for programmatic use).

Name	Vorgabe
text	—
language	"de"
profile	"makecode"

13.8 raw-blockly

```
raw-blockly(..args)
```

Enable Blockly code blocks in raw text:

```
#show: raw-blockly()
```blockly
wiederhole (4) mal:
ende
```
```

A `jwinf` fence uses the `jwinf` profile whatever the arguments say, a `jwinf-turtle` fence the turtle sandbox's colours.

13.9 raw-makecode

```
raw-makecode(..args)
```

Enable MakeCode code blocks in raw text:

```
#show: raw-makecode()
```makecode
beim Start
 zeige Zahl (0)
ende
```
```

A `microbit` fence is the same; a `calliope` fence uses the Calliope mini profile whatever the arguments say.

13.10 raw-scratch

```
raw-scratch(..args)
```

Enable scratch code blocks in raw text:

```
#show: raw-scratch()
```

With language: `#show: raw-scratch(language: "de")`.

13.11 scratch

```
scratch(
  text,
  language: auto,
  theme: auto,
  scale: auto,
  font: auto,
  line-numbering: auto,
  line-numbers: auto,
  line-number-start: auto,
  line-number-first-block: auto,
  line-number-gutter: auto,
  inset-scale: auto,
  colors: auto,
)
```

Render scratch blocks from text. Works standalone or inside `#blockst[...]`. Supports 26 languages: en, de, fr, es, it, pt, nl, pl, ru, ja, ...

| Name | Vorgabe |
|-------------------------|---------|
| text | – |
| language | auto |
| theme | auto |
| scale | auto |
| font | auto |
| line-numbering | auto |
| line-numbers | auto |
| line-number-start | auto |
| line-number-first-block | auto |
| line-number-gutter | auto |
| inset-scale | auto |
| colors | auto |

13.12 scratch-execute

```
scratch-execute(text, language: "en")
```

Execute scratch text, producing scratch-run commands.

| Name | Vorgabe |
|----------|---------|
| text | – |
| language | "en" |

13.13 scratch-labels

```
scratch-labels(text, language: "en")
```

Parse scratch text and return a dictionary mapping `#labels` to rendered line numbers.

| Name | Vorgabe |
|------|---------|
| text | – |

| Name | Vorgabe |
|----------|---------|
| language | "en" |

13.14 scratch-parse

```
scratch-parse(text, language: "en")
```

Parse scratch text to AST (for programmatic use).

| Name | Vorgabe |
|----------|---------|
| text | – |
| language | "en" |

13.15 set-blockst

```
set-blockst(
  theme: none,
  profile: none,
  colors: none,
  scale: none,
  stroke-width: none,
  font: none,
  line-numbering: none,
  line-numbers: none,
  line-number-start: none,
  line-number-first-block: none,
  line-number-gutter: none,
  inset-scale: none,
  language: none,
)
```

Global settings applied to all scratch() and sb3 calls.

colors lays the document's own category colours over the profile's palette: set-blockst(colors: (logik: "#73cc47")) — a hex string or a Typst colour per category. The shades a theme derives (bevel, stroke, high-contrast, grayscale) follow the new fill.

| Name | Vorgabe |
|-------------------------|---------|
| theme | none |
| profile | none |
| colors | none |
| scale | none |
| stroke-width | none |
| font | none |
| line-numbering | none |
| line-numbers | none |
| line-number-start | none |
| line-number-first-block | none |
| line-number-gutter | none |
| inset-scale | none |
| language | none |